Deliverable 1: Initial Planning and Design

Prateek Sahay and Venkata Sai Makani

University of the Cumberlands

MSCS632-M30: Advanced Programming Languages

Jay Thom

October 2, 2026

Deliverable 1: Initial Planning and Design

Project Purpose and Scope

Our two-member team will implement the Expense Tracker Application independently in Python and C++17. The application will help a user record everyday expenses, find relevant records, and understand spending by category. Both implementations will provide the same command-line interface and use the same portable CSV storage contract. The project will use this common problem to examine how programming languages influence data representation, validation, resource management, error handling, and code organization.

The required functions are adding an expense with a date, amount, category, and description; viewing recorded expenses; filtering by an inclusive date range or an exact category; searching category and description text; and calculating category and overall totals. We also plan a small delete-by-ID extension. A graphical interface, network service, authentication, and concurrent writers are outside the planned scope. A focused, single-user interface will allow the team to invest time in reliable behavior and meaningful comparisons of our own source code.

Application Structure and Shared Design

Each language implementation will stand alone: the C++ program will not call Python, and the Python program will not require the C++ executable. Both will expose add, list, search, summary, and delete commands. A --data option placed before the command will select the CSV file; the default will be data/expenses.csv relative to the working directory. Help will describe required arguments. Successful operations will return status 0, while invalid input or storage failures will return status 2 with an Error message on standard error.

The shared CSV header will be id,date,amount_cents,category,description. Every expense will have a unique positive ID, a calendar-valid YYYY-MM-DD date, an exact positive

integer amount in cents, and nonempty text fields. Category and description will use printable ASCII with maximum lengths of 40 and 200 characters. Surrounding spaces in new text will be trimmed; controls and newlines will be rejected. Commas and double quotes will be preserved through standard CSV quoting. Existing files will be validated completely before a change is saved.

Amounts will be parsed from decimal strings with at most two fractional digits. For example, 12.34 will become 1234 cents, avoiding binary floating-point rounding. The maximum individual expense will be $999999999.99. IDs will be limited to 2147483647, records to 100000, and accumulated totals to the signed 64-bit range. IDs will use the largest existing value plus one. Filtering will include both date endpoints. Category matching will be case-sensitive, while search will ignore ASCII case. Lists will preserve insertion order, and summaries will sort category names.

Each save will write a complete temporary file in the target directory and replace the original after writing succeeds. This design helps prevent a partially written destination during an ordinary save failure. It does not provide simultaneous-writer coordination. The applications will therefore document that only one process should modify a given data file at a time.

Component	Python implementation	C++ implementation
Command-line interface	python/main.py: argparse, command dispatch, formatted results	cpp/main.cpp: option parsing, command dispatch, formatted results
Records and business rules	python/tracker.py: dictionaries, validation functions, filtering and summaries	cpp/tracker.hpp and cpp/tracker.cpp: Expense struct, Tracker class, filters and summaries
Persistence and validation	Python csv module, datetime.date, temporary-file replacement	CSV parser, calendar checks, file streams and platform-aware replacement
Build and verification	Python 3.10+; python/test_tracker.py	C++17 compiler; cpp/CMakeLists.txt

Component	Python implementation	C++ implementation
Shared acceptance testing	tests/test_tracker.py exercises real Python and C++ commands	The same fixtures verify behavior and CSV exchange in both directions

Expected Language Differences and Challenges

Python will use dictionaries for expense records, with TypedDict and other type hints documenting expectations. Its runtime remains dynamically typed, so explicit validation will protect user input and loaded files. C++ will use a statically typed Expense struct and a vector of records. We expect C++ declarations to require more source code but to make field types and interfaces visible to the compiler.

Python's datetime and csv libraries should reduce the amount of infrastructure code. The C++17 implementation will require explicit calendar validation and a small CSV parser because this design does not introduce an external library. We will compare that added work with the control it provides over accepted input. Python comprehensions and dictionary accumulation will be compared with C++ loops, standard containers, and member functions.

Memory and resource ownership will also differ. Python manages object lifetimes through its runtime and will use context managers for open files. C++ will use value ownership, standard containers, scoped streams, and a temporary-file cleanup object rather than manual new/delete. Both versions will use exceptions to pass errors to the command-line boundary. Concurrency will not be added solely to demonstrate a feature: overlapping writes would require locking beyond this single-user design. Any performance discussion will distinguish expectations from measurements.

Proposed Task Assignment and Collaboration

Prateek Sahay will primarily own the Python implementation and integration checklist. His prior robotics integration work supports coordinating interfaces, investigating failures across components, and organizing reproducible tests. Venkata Sai Makani will primarily own the C++ implementation and build instructions. This is a proposed allocation for balanced ownership; his specific strengths have not yet been provided, so the team should confirm the assignment together rather than treating C++ experience as an established fact.

Both members will agree on the data contract, review the other implementation, run shared acceptance cases, contribute to the reports, and present their source-code examples. The more verbose C++ validation and storage work will be balanced against Python's shared testing and integration work. Documentation and presentation tasks will be divided after the first implementation milestone to adjust for the actual workload.

Member	Primary ownership	Shared responsibilities
Prateek Sahay	Python CLI/core; shared test fixtures and integration checklist	Review C++ behavior; coauthor reports; demonstrate Python and language tradeoffs
Venkata Sai Makani	C++ CLI/core; compilation and storage implementation	Review Python behavior; coauthor reports; demonstrate C++ and language tradeoffs
Both members	Shared CSV/CLI contract and acceptance decisions	Peer review, debugging, README review, demonstration rehearsal and final submission

Three-Day Timeline and Acceptance Milestones

The schedule below is the initial residency plan for Friday, October 2 through Sunday, October 4, 2026. It defines reviewable milestones rather than claiming that future work or individual contributions have already occurred.

Day	Planned work	Milestone and acceptance criteria

Day	Planned work	Milestone and acceptance criteria
Friday, October 2	Agree on features and storage/CLI contracts; assign owners; establish repository and branch structure; draft component design	Deliverable 1 ready; both members can explain responsibilities, record fields, validation rules and the next milestone
Saturday, October 3	Implement add/list/filter/search/summary in both languages; add persistence and deletion; build C++; run shared tests and capture demonstrations	Core commands work independently; exact totals and inclusive filtering agree; CSV can move between implementations; significant defects resolved; Deliverable 2 includes actual screenshots
Sunday, October 4	Complete final fixes and documentation; verify parity; write APA 7 comparison; prepare slides and rehearse a 15–20 minute presentation	All required checks pass; README reproduces runs; comparison contains source-code evidence; both members participate; written report and slides ready

Repository and Review Workflow

The proposed public repository is cross-language-expense-tracker. Repository link: https://github.com/psahay101/cross-language-expense-tracker. Repository creation has been verified. Both source trees, the shared tests, README, and deliverable documentation will be kept together so the comparison can refer to concrete code.

The team will use a stable main branch and small feature branches, such as feature/python-core and feature/cpp-core. Each change will identify the feature or defect it addresses. The other member will review behavior, validation, and readability before integration. Shared tests will run after changes that affect the data format or CLI. The README will document prerequisites, build commands, sample usage, validation rules, and the single-writer limitation. No collaborator invitation or contribution history will be assumed until the team has actually established access.